CURSO DE C AVANZADO PARA LINUX

1) **PUNTEROS**: prueba con los punteros a función: hay código en practicas/codigo/punteros_funcion.c

    a. Funciones que reciben un puntero a función
    b. Funciones que devuelven un puntero a función

    c. Utilizar punteros a función parametrizar el criterio en una función de ordenación.

    d. Utilizar punteros a función para pasar un método de ordenación a otra función y ordenar.

    e. Probar la función qsort de stdlib.h para pasar un puntero a función con el criterios de ordenación:
    void **qsort**(void *base, size_t number_elements,
    size_t element_size, int (*compare)(const void *first, const void* second))

2) **PUNTEROS**: Probar a recorrer los arrays de int / char con punteros, acceso dirección indirecto, mostrar las direcciones de memoria.

3) **PUNTEROS CTES Y PUNTEROS A CTES.**

4) **PUNTEROS : CALLBACKS / ALINEAMIENTOS**
    a. Utilizar arrays de punteros a función, buscar y devolver la dir de la función. Uso de typedef para la definir de una forma más cómoda los callback.
    b. Definir otra estructura para el contexto para pasar por parámetro al callback (se pasa como un puntero void).
    c. Pruebas con aliasing: con tipos compatibles, distintos tipos y uso de restrict (promesa al compilador que no vamos a pasar la mismadir. De memoria en ambos parámetros, si le pasamos

la misma dirección entramos en comportamiento indefinido del compilador).

d. Alineamiento. Pruebas con arrays de chars y copiar a uint32_t con direcciones que no son múltiplo de 4. y usar memcpy(destino, origen, cantidad). Copia los 4 bytes de forma alineada.

5) **PUNTEROS A FUNCION:**

Desarrollar una aplicación en C, ejecutada en Ubuntu/Lubuntu, que implemente una **Máquina de Estados Finitos (FSM)** capaz de reaccionar a distintos **eventos** y cambiar de estado según la lógica definida. La FSM debe estar completamente estructurada utilizando:

- un **enum de estados**,
- un **enum de eventos**,
- una **estructura de contexto**,
- una **tabla de handlers**,
- una **función por estado**,
- una **función de despacho de eventos**,
- y un **bucle principal** que procese eventos simulados.

Definir un conjunto de estados que representen el ciclo de vida de un sistema simple:

- IDLE, RUNNING, ERROR

Definir un conjunto de eventos que afecten a la FSM:

- START, STOP, FAIL

Mantener un contexto, que incluya al menos:

- El estado actual,
- un contador o dato auxiliar (opcional).

Implementar una función manejadora por cada estado, que reciba:

- el contexto (fsm_t *),

- el evento (event_t), y devuelva el siguiente estado.

Crear una tabla de handlers, indexada por estado, que permita saltar directamente a la función correspondiente sin usar switch.

Implementar una función de despacho de eventos, que:

- lea el estado actual del contexto,
- invoque el handler adecuado,
- actualice el estado del contexto.

Simular una secuencia de eventos, por ejemplo:

START → FAIL → START → STOP → START → STOP

Ejecutar la FSM en un bucle, procesando cada evento y mostrando:

- el estado actual,
- el evento recibido,
- el estado resultante.

Salida del programa:

[Estado actual], evento recibido, y el nuevo estado.

[IDLE] Evento: START → Nuevo estado: RUNNING

[RUNNING] Evento: FAIL → Nuevo estado: ERROR

[ERROR] Evento: START → Nuevo estado: RUNNING

[RUNNING] Evento: STOP → Nuevo estado: IDLE

[IDLE] Evento: START →Nuevo estado: RUNNING

[RUNNING] Evento: STOP → Nuevo estado: IDLE

6) **MODULOS, EXTERN, STATIC** con guards / pragma. Acceso global y local
   a. Modulo de configuración. Definir en el .H funciones públicas y una variable extern.
   b. En el .C declarar la variable global, añadir alguna static global.

7) Igual que el anterior pero probar con **#pragma once**

7) MEMORIA: prueba con malloc, calloc y realloc. Para cargar un fichero, por lineas, cada en un array. Estudiar posibles, mejoras de doblar el número de filas.

5) MEMORIA: Pruebas con errores intencionados sobre memoria.

a) liberar dos veces el mismo bloque.

b) Acceder a un bloque después de liberarlo

c) Devolver la dirección de una variable local

d) Overflow en un array del heap

e) Leer antes de la posición de comienzo del bloque

f) Fuga de memoria, un bloque sin liberarlo

g) Pérdida del bloque original si realloc falla (el puntero que devuelve es NULL)

h) Overflow en un string, copiar una cadena demasiado grande para el bloque reservado.

i) Liberar memoria que no fue reservada.

j) Utilizar una posición no inicializada de un bloque de memoria

COMPILACION: gcc -std=c11 -Wall -Wextra -Wpedantic -g -O0 errores_memoria.c -o errores_memoria

VALGRIND: valgrind \

```
--leak-check=full \
--show-leak-kinds=all \
--track-origins=yes \
./errores_memoria
```

EJECUCIÓN CON ADDRESSSANITIZER

gcc \

-std=c11 \
-Wall \
-Wextra \
-Wpedantic \
-g \
-O0 \
-fsanitize=address,undefined \
-fno-omit-frame-pointer \
errores_memoria.c \
-o errores_memoria

Cada herramienta detecta una cosa distinta:

- Double free: Valgrind y AddressSanitizer.
- Use after free: Valgrind y AddressSanitizer.
- Dirección de variable local: compilador y AddressSanitizer.
- Acceso fuera de límites: Valgrind y AddressSanitizer.
- Fuga de memoria: Valgrind y LeakSanitizer.
- Uso de memoria no inicializada: especialmente Valgrind.
- free() inválido: Valgrind y AddressSanitizer.
- Uso inseguro de realloc(): Valgrind detectará la fuga si realloc() falla.

PARÁMETROS UTILIZADOS EN LA COMPILACIÓN

gcc -std=c11 -Wall -Wextra -Wpedantic -g -O0 errores_memoria.c -o errores_memoria

gcc

Ejecuta el compilador de C de GNU. CC realiza normalmente el preprocesado, la compilación, el ensamblado y el enlazado para generar el ejecutable.

-std=c11

Compila utilizando el estándar C11. Permite usar las características definidas por la versión de C publicada en 2011 y ayuda a detectar construcciones no compatibles con ese estándar.

-Wall

Activa un conjunto amplio de advertencias habituales. No activa literalmente todas las advertencias existentes, a pesar del nombre. Puede detectar variables no utilizadas, formatos incorrectos en printf(), funciones sin declarar y otros problemas frecuentes.

-Wextra

Activa advertencias adicionales que no están incluidas en -Wall. Por ejemplo, puede advertir de parámetros no utilizados y determinadas comparaciones sospechosas.

-Wpedantic

Solicita advertencias cuando el código utiliza extensiones o construcciones que no cumplen estrictamente el estándar seleccionado mediante -std=c11.

Es útil para escribir código C más portable.

-g

Añade al ejecutable información de depuración. Permite que herramientas como GDB y Valgrind muestren nombres de variables, nombres de funciones y números de línea.

-O0

Desactiva las optimizaciones del compilador. La letra es una O mayúscula, no el número cero. El nivel 0 facilita la depuración porque el código ejecutado se parece más directamente al código fuente y las variables no se eliminan ni reorganizan agresivamente.

Nombre del fuente.c y -o nombre salida ejecutable, si no se indica le llama a.out

Ejecución del programa:

./errores_memoria

Compilación recomendada durante el desarrollo:
gcc -std=c11 -Wall -Wextra -Wpedantic -g -O0 errores_memoria.c -o errores_memoria

Compilación tratando todas las advertencias como errores:
gcc -std=c11 -Wall -Wextra -Wpedantic -Werror -g -O0 errores_memoria.c -o errores_memoria **-Werror**

Convierte las advertencias del compilador en errores. Si existe una advertencia, no se genera el ejecutable.

6) PRÁCTICA MEMORIA: PATRÓN STATE CON CALLBACKS EN C

Desarrollar en C una aplicación que simule un gestor de descargas mediante una máquina de estados y el patrón de diseño State.

La descarga podrá encontrarse en uno de los siguientes **estados**:

- PREPARADA
- DESCARGANDO
- PAUSADA
- FINALIZADA
- ERROR

La aplicación deberá procesar los siguientes **eventos**:

- INICIAR
- AVANZAR
- PAUSAR
- REANUDAR
- FALLAR
- CANCELAR
- REINICIAR

COMPORTAMIENTO

- PREPARADA: permite iniciar o cancelar la descarga.

- DESCARGANDO: permite avanzar, pausar, cancelar o provocar un error.
- PAUSADA: permite reanudar o cancelar. El evento AVANZAR deberá ignorarse.
- FINALIZADA: estado terminal alcanzado cuando el progreso llega al 100 %.
- ERROR: permite reiniciar la descarga desde cero.

Cada evento AVANZAR incrementará el progreso de la descarga en un 25 %.

La solución deberá utilizar:

- Enumeraciones para representar los estados y los eventos.
- Una estructura de contexto con el estado actual, el porcentaje descargado y el código de error.
- Una estructura común para representar el comportamiento de los estados.
- Callbacks on_enter, on_event y on_exit.
- Punteros a función para implementar los callbacks.
- Una función para despachar eventos.
- Una función común para realizar las transiciones entre estados.

SECUENCIA DE PRUEBA

La aplicación deberá procesar la siguiente secuencia:

- INICIAR
- AVANZAR
- AVANZAR
- PAUSAR
- AVANZAR
- REANUDAR
- AVANZAR
- AVANZAR

El evento AVANZAR recibido mientras la descarga está en estado PAUSADA deberá ignorarse.

RESULTADO ESPERADO

La aplicación deberá mostrar:

- El estado actual.
- El evento recibido.
- Las transiciones realizadas.
- La ejecución de los callbacks de entrada y salida.
- El porcentaje descargado.
- Los eventos ignorados o no permitidos.

La ejecución deberá terminar cuando la descarga alcance el estado FINALIZADA.

4) SOCKETS: probar los distintos modelos de comunicación. De 1 a 1, 1 a N, utilizando hilos y procesos.

5) SOCKETS: Con el modelo de c**onexión 1 a 1**, implementar un cliente y un servidor para enviar un fichero del cliente al servidor.

Hay que enviar estos datos:

- longitud del nombre del fichero.

- el nombre del fichero

- tamaño del contenido del fichero

- contenido del fichero.

El una vez recibidos o enviados los 3 primeros parámetros se envía o se recibe el contenido del fichero en bloques de 1024 bytes (unsigned char), hasta que se termine. Tener en cuenta que último bloque será más pequeño de 1024 bytes. Tenemos el fichero CSV: “pedidos_final.csv” para poder enviar un fichero, aunque el fichero puede ser binario (que es como se tratará en la práctica).